Proxy Design Pattern

> **GoF intent:** *Provide a surrogate or placeholder for another object to control access to it.* The client talks to a stand-in (the proxy) that implements the same interface as the real object, so the client cannot tell the difference — and the proxy gets a chance to control access (lazy loading, logging, caching, guarding) before/after forwarding the call.

Structure of this implementation

GoF role	Class in this module
Subject (common interface)	`Image` — declares `display()`
RealSubject (expensive, does the work)	`RealImage` — loads from disk in its constructor, then displays
Proxy (virtual proxy, controls access)	`ProxyImage` — holds the file name; instantiates `RealImage` on first `display()`
Client	`ProxyDesignPattern.main()` — programs against `Image`

How it works

1. `Image` is the contract both the real object and the surrogate share — because `ProxyImage implements Image`, any code written against `Image` accepts the proxy transparently.
2. `RealImage` is deliberately expensive: its **constructor** simulates loading the file from disk.
3. `ProxyImage` stores only the cheap file name. On the **first** `display()` call it creates the `RealImage` (paying the load cost), caches it, and delegates; on every later call it delegates straight to the cached instance — no reload.

4. The client creates only the proxy. If `display()` is never called, the expensive object is never built at all.

Verified output (`java com.design.patterns.proxy.ProxyDesignPattern`):

```
Proxy Design Pattern
----Calling via proxy
-----Loading image from disk: holiday-photo.png
-----Displaying real image: holiday-photo.png
----Calling via proxy
-----Displaying real image: holiday-photo.png
```

Note the proof in the output: two `display()` calls, but `Loading image from disk` appears only once — the proxy deferred creation to first use and reused the loaded subject afterwards.

Why this follows the pattern

- ✅ Proxy and real subject share one interface (`Image`) — the client is substitution-safe and is typed to the interface.
- ✅ The proxy **controls access**: it decides *when* the real subject comes into existence (virtual proxy) and intercepts every call (logging seam).
- ✅ Composition + forwarding: the proxy holds a reference and delegates, it does not inherit implementation.
- ✅ Same shape as real-world proxies: Hibernate lazy-loading entities, Spring AOP proxies, `java.lang.reflect.Proxy` .

History

The first version had the packages swapped (`RealImage` in the `proxy` package, `ProxyImage` in the `realsubject` package), made the *client* construct the `RealImage` eagerly, and only added a log line. Fixed on 2026-07-08: packages match roles, the proxy now lazily creates and caches the real subject (true virtual proxy), and the client is typed to `Image` .